

Sri Lanka Institute of Information Technology

Faculty of Computing

Prof.Sanvitha Kasthuriarachchi

IT2130 - Operating Systems and System Administration

Year 02 and Semester 02

Lecture 09

Introduction to Deadlock

Deadlocks

- Several processes may compete for a finite number of resources, and some of them may wait for the resources forever because the resources are held by other waiting processes.



System Model

- Resources are partitioned into several types, each consists of some number of identical instances.
 - **Identical**: allocation of any instance of the type will satisfy process's request.
 - Resources may be physical resources (printers, tape drives, CPU cycles), or logical resources (files, semaphores, and monitors).
 - A **pre-emptible** resource is one that can be taken away from a process with no ill effect to the process; e.g., memory.
 - A **non-preemptible** resource is one that cannot be taken away from its user since it will make the user fails; e.g., printers.
 - In general, potential deadlocks involve this resource type.

System Model

- Each process uses a resource as follows:
 - **Request** the resource; a process must wait if the resource is being used by another process.
 - **Use** the resource; e.g., the process can print on the printer.
 - **Release** the resource.

Necessary Conditions for Deadlock

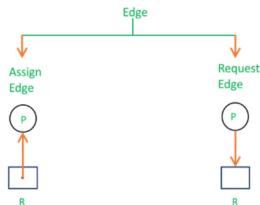
Four conditions must hold for a deadlock to occur:

- ➊ **Mutual exclusion condition.** Only one process at a time can use the resource.
- ➋ **Hold and wait condition.** A process holding at least one resource is waiting to acquire additional resources held by other processes.
- ➌ **No pre-emption condition.** A resource can be released only voluntarily by the process holding it after that process has completed its task.
- ➍ **Circular wait condition.** There exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource held by P_1 , P_1 for one held by P_2 , $\dots$, and P_n is waiting for a resource held by P_0 .

Note: the four conditions are not completely independent, e.g., the circular-wait condition implies the hold-and-wait condition.

Deadlock Modelling

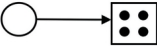
- Deadlocks can be described more precisely in terms of a directed graph $G(V, E)$. This is called *System resource-allocation graph*.
- V is partitioned into two types:
 - Set of processes in the system:
 $P = \{P_1, P_2, \dots, P_n\}$
 - Set of all resource types in the system:
 $R = \{R_1, R_2, \dots, R_n\}$
- Request edge* – directed edge $P_i \rightarrow R_j$
 - Process P_i requests an instance of resource R_j
- Assignment edge* – directed edge $R_j \rightarrow P_i$
 - An instance of resource R_j has been allocated to process P_i



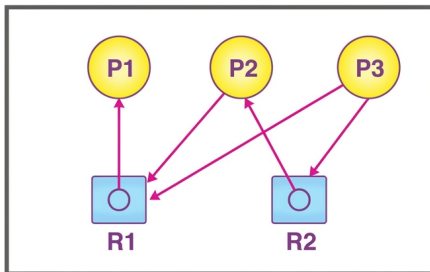
Model Symbols

- Process: ○

Resource type with 4 instances: 

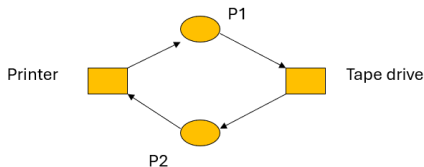
- P_i requests R_j : 
 R

- P_i uses R_j : 
 R

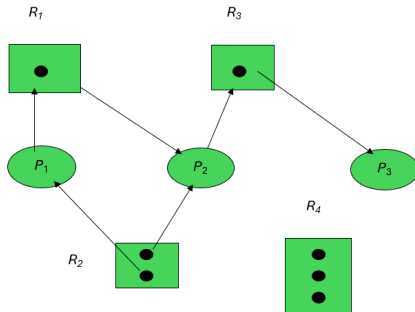


Resource Allocation Graphs and Deadlocks

- If the graph contains **no cycles**, no process in the system is deadlocked.
- If the graph contains **a cycle**, deadlock may exist:
 - If each resource type has **one instance**, cycle means deadlock.
 - If each resource type has **several instances**, cycle is necessary but **not sufficient condition for deadlock**.

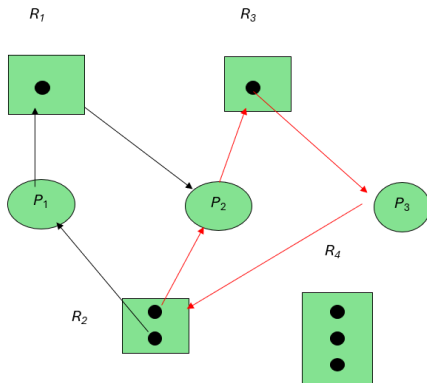


Example 1



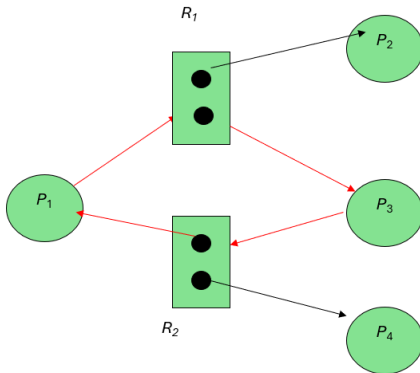
- A resource allocation graph with no cycles. Therefore, no deadlocks.

Example 2



- A resource allocation graph with a cycle and a deadlock.

Example 3



- A resource allocation graph with a cycle but no deadlock.
- P_4 can release R_2 which gets allocated to P_3 , breaking the cycle.

Three Methods for Handling Deadlock

- Use a protocol to ensure that the system will never reach deadlock
 - Using deadlock prevention and/or deadlock avoidance techniques
- Allow the system to enter a deadlock state and then recover
 - Needs deadlock detection and deadlock recovery algorithms
- Ignore the problem and pretend that deadlocks never occur in the system
 - Used by most OSs, including UNIX
 - Also called the **ostrich** algorithm

Deadlock Prevention

Use a set of methods to ensure that **any one** of the four deadlock conditions cannot hold:

① Deny mutual exclusion

- Processes are sharing the resources.

② Deny hold and wait

- Must guarantee that whenever a process requests a resource, it does not hold any other resources.
- Each process is granted all resources before it starts.
- If a process needs more resources, release all resources before requesting new ones.

Deadlock Prevention

③ Prevent no pre-emption (i.e., allow pre-emption)

- When a process holding some resources requests other resources that cannot be immediately allocated, it must release all resources currently being held.

④ Deny circular wait

- All resource types are ordered, e.g.,
 - $F(\text{card reader}) = 1$ $F(\text{disk drive}) = 5$
 - $F(\text{tape drive}) = 7$ $F(\text{printer}) = 12$
- Each process must request resources in increasing order.

Deadlock Avoidance

- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that **there can never be a circular-wait condition**.
- A resource-allocation state is defined by:
 - The number of **available** and **allocated** resources, and
 - The **maximum demands** of the processes.

Safe State

- When a process requests an available resource, the system checks if its allocation keeps the system in **safe state**.
- The system is in safe state if there exists a **safe sequence** of all processes.
- A sequence $\langle P_1, P_2, \dots, P_n \rangle$ is safe if, for each P_i , the resources requested by P_i can be allocated from the currently available resources + resources held by all P_j , with $j < i$.
 - If P_i 's resource needs are not immediately available, P_i waits until all P_j have finished.
 - When all P_j are finished, P_i obtains the needed resources, executes, returns the allocated resources, and terminates.
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Example

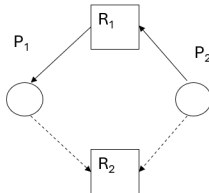
Consider a system with 12 resources of the same type, and 3 processes:

	<u>Maximum needs</u>	<u>Allocation</u>	<u>Current need</u>
P_0	10	5	5
P_1	4	2	2
P_2	9	2	7

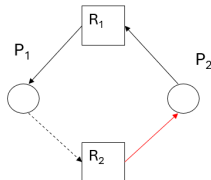
- At time t_0 , **available resource** = 3, and the system is in safe state.
- There is a safe sequence $\langle P_1, P_0, P_2 \rangle$.
- What if at t_1 one more resource is allocated to process P_2 ?
 - The system is in unsafe state
 - deadlock can occur.

Resource-Allocation Graph Algorithm

- **Claim edge** $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j . It is represented by a dashed line.

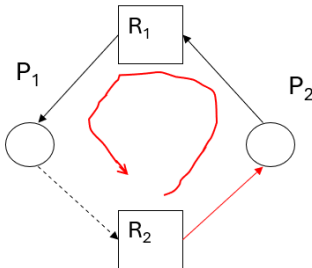


- **Claim edge** converts to a **request edge** when a process requests a resource.



Example

- When a resource is released by a process, assignment edge converts to a claim edge
- Resources must be claimed a priori in the system
- Need a cycle detection algorithm.



- This algorithm can only be used for systems comprising resource types with single instance.

Banker's Algorithm

- The algorithm for a system comprising resource types with **multiple instances**.
- Each process must *a priori* claim maximum number of instances of each resource type that it may need.
- When a process requests a resource:
 - It may have to wait (if resource allocation may lead to unsafe state) until some other process releases enough resources.
- When a process gets all its resources:
 - It must return them in a finite amount of time.

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ; 3 resource types A (10 instances), B (5 instances), and C (7 instances).
- Snapshot at time T_0 :

	<i>Allocation</i>			<i>Max</i>			<i>Available</i>			<i>Need</i>		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2	7	4	3
P_1	2	0	0	3	2	2				1	2	2
P_2	3	0	2	9	0	2				6	0	0
P_3	2	1	1	2	2	2				0	1	1
P_4	0	0	2	4	3	3				4	3	1

- The content of matrix *Need* is defined to be $Max - Allocation$.
- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies the safety criteria.

Example (P_1 requests $(1, 0, 2)$)

- Check that $request \leq need$ (that is, $(1, 0, 2) \leq (1, 2, 2)$) $\rightarrow$ **true**
- Check that $request \leq available$ (that is, $(1, 0, 2) \leq (3, 3, 2)$) $\rightarrow$ **true**

Before Adjustment

	Allocation		
	A	B	C
P_0	0	1	0
P_1	2	0	0
P_2	3	0	2
P_3	2	1	1
P_4	0	0	2

After Adjustment

	Alloc.			Need			Avail.		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	4	3	2	3	0
P_1	3	0	2	0	2	0			
P_2	3	0	2	6	0	0			
P_3	2	1	1	0	1	1			
P_4	0	0	2	4	3	1			

- $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ or $\langle P_1, P_4, P_3, P_0, P_2 \rangle$ satisfies safety requirement.
- Can request for $(3, 3, 0)$ by P_4 be granted? $(0, 2, 0)$ by P_0 ?

Deadlock Detection

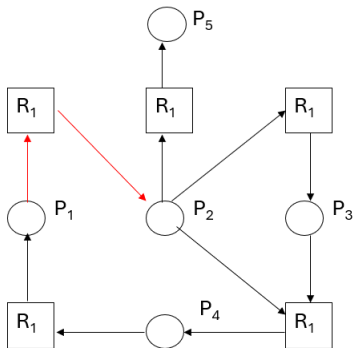
- If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation may occur.
- Need a **deadlock detection algorithm** that examines the state of the system to determine whether a deadlock has occurred.
- Need a recovery algorithm to recover from deadlock.

Deadlock detection for **single instance** of each resource type:

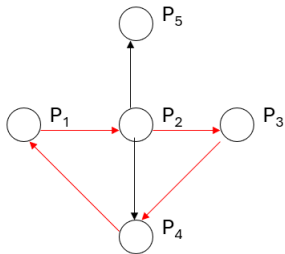
- Maintain a wait-for graph.
- Periodically invoke an algorithm that searches for a cycle in the graph.

Example

Resource Allocation Graph



Wait for Graph



Deadlock Recovery

1) Terminate processes

- Kill (abort) all deadlocked processes.
- Kill one process at a time until deadlock cycle is eliminated.
- In which order should we choose a process to abort?
 - The process with lowest priority
 - How long the process has computed, and how much longer to completion
 - Resources the process has used
 - Resources the process needs to complete
 - How many processes will need to be terminated
 - Is process interactive or batch?
- **Problem:** what if the process is in the middle of updating a file?
Aborting the process may lead to incorrect file.

Deadlock Recovery

2) Pre-empt a resource from a process

- How to select a victim (process) to minimize cost?
- Roll back the process to some safe state and restart from there.
 - How do we find a safe state?
 - Easiest way: destroy the process and restart.
 - Use checkpoints during execution.
- Starvation – same process may always be picked as victim.
 - How do we ensure no starvation?
 - Include number of rollbacks in cost factor.

End of Lecture 9

Thank You